An Event-Driven Microservices Home Energy Tracker with Gateway Authentication and ML-Based Anomaly Detection

Rohit Shrivastava, Anshika Rai, Harsh Gupta, and Syamasudha Veeragandham

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India

rohit.lalbahadur2023@vitstudent.ac.in; anshika.rai2023@vitstudent.ac.in; harsh.gupta2023@vitstudent.ac.in; syamasudha.v@vit.ac.in

Abstract—Residential energy applications require a verifiable path from telemetry ingestion to historical storage, user-facing analysis, and notification, while preserving access control at the system boundary. This paper presents an event-driven microservices Home Energy Tracker evaluated in a local Docker Compose environment. The implementation routes Wokwi-compatible ESP32 telemetry through an API gateway with Keycloak JWT validation, resource-ownership enforcement, and a machine-token path. Ingestion events are distributed through Apache Kafka to time-series usage storage, deterministic insight functions, an alert service with a 15-minute deduplication policy, and a persisted Random Forest anomaly-inference service. The offline model was evaluated on a 5,000-record CSV using a chronological 70/15/15 split; on the held-out interval it obtained precision 0.88095, recall 0.92500, F1-score 0.90244, ROC-AUC 0.99743, and PR-AUC 0.94853. Local validation recorded expected outcomes for 8/8 security cases, 450/450 accepted k6 ingestion requests with 109.77 ms p95 response time, and a controlled end-to-end synthetic run in which analytics became available in 3.79 s and a threshold alert was captured by Mailpit in 13.64 s. These observations establish local integration and evaluation evidence, not physical-meter accuracy, external ML generalization, production capacity, or comprehensive fault tolerance.

Index Terms—*home energy monitoring, microservices, Apache Kafka, anomaly detection, Keycloak, JWT, Wokwi, Prometheus.*

I. INTRODUCTION

Residential energy bills provide limited visibility into the timing, distribution, and abnormality of household electricity consumption. Existing IoT studies demonstrate low-cost sensing and visualisation, while other work considers anomaly detection, appliance-level analysis, and distributed energy-management architectures [1]–[24]. A useful engineering contribution in this space is therefore not a generic claim of intelligence or scalability, but a traceable and testable integration of telemetry, storage, analysis, alerting, and access control.

The present work revises an earlier functional prototype into an evidence-bounded final implementation. It combines a Wokwi ESP32 simulator, Spring Boot services, FastAPI-based ML inference, Apache Kafka, InfluxDB, MySQL, Keycloak, Prometheus, Grafana, a React dashboard, and Mailpit. The system is deliberately evaluated under local Docker Compose conditions. The simulator and controlled REST submissions use the same ingestion contract, but neither substitutes for calibrated electricity-meter validation.

A. Objectives and Contributions

The final objectives were to: (i) implement an event-driven telemetry-to-storage path; (ii) enforce gateway-level authentication, authorization, ownership, and device-token controls; (iii) provide deterministic forecasting, ranking, standby estimation, and deduplicated alerts; (iv) train, evaluate, persist, and integrate a supervised anomaly classifier; and (v) collect bounded local evidence from security, functional, performance, observability, and controlled fault scenarios.

- An implemented 17-service local architecture that separates identity, telemetry ingestion, event processing, time-series usage, metadata, deterministic insights, ML inference, alerting, and monitoring.
- An offline chronological evaluation of a persisted Random Forest model against a Logistic Regression comparator, reported separately from simulator and runtime integration evidence.
- A reproducible local validation record covering gateway security cases, dashboard-route regression checks, alert deduplication, Prometheus targets, a k6 ingestion scenario, two controlled service-stop scenarios, and an end-to-end synthetic workflow.

II. RELATED WORK AND RESEARCH GAP

A. Existing Approaches

Low-cost monitoring systems based on ESP32-class controllers, smart meters, and IoT dashboards motivate the telemetry interface used in this study [3]–[6], [13], [16], [18]–[21], [23]. Research on anomaly detection and appliance-aware analysis illustrates the value of labelled time series and careful evaluation [1], [7]–[11], [15], [22]. Microservice and cloud-oriented approaches further motivate service separation and event-mediated processing [2], [12], [14], [17], [24]. These directions are complementary: sensing work does not by itself validate software architecture, and architectural work does not automatically establish valid energy-model performance.

B. Revised Research Gap

The original gap concerned the absence of a concise, inspectable linkage between household telemetry, event-driven services, time-series history, transparent energy insights, and notification. The completed system now addresses this integration gap more broadly by adding an evaluated supervised classifier, gateway-based identity controls, deduplicated alerting, and local operational evidence. It only partially addresses the surrounding research gaps: the source and label-generation process of the training data are undocumented; Wokwi inputs are synthetic; evaluation is single-host and short-duration; and physical-meter, multi-household, utility-bill, and production-scale validation remain unresolved.

III. FINAL SYSTEM ARCHITECTURE AND METHODOLOGY

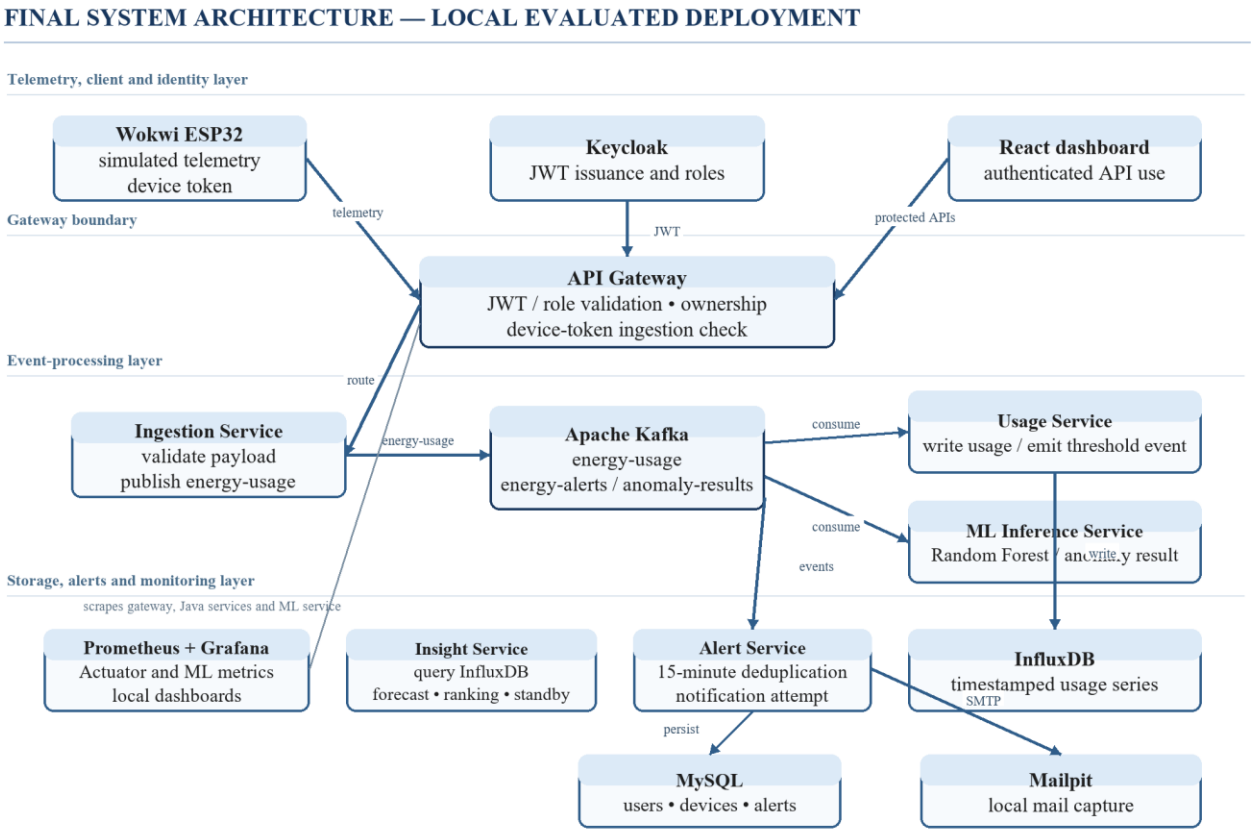


Fig. 1. Final architecture of the locally evaluated Home Energy Tracker. Solid arrows denote implemented request or event flow; Prometheus/Grafana represent monitoring rather than a data-processing dependency.

A. Telemetry, Gateway, and Event Processing

The Wokwi ESP32 simulator constructs a JSON payload containing deviceId, energyConsumed, and timestamp and posts it to the simulator-compatible ingestion endpoint. Gateway access is separated by caller type. User-facing API routes require Keycloak-issued JWTs and are subject to role mapping and path-based resource-ownership checks. The telemetry endpoint accepts either an authenticated JWT or a configured X-Device-Token. The gateway does not establish production-grade security; it provides the evaluated local boundary controls described in Section V-C.

The Ingestion Service validates the telemetry contract and emits an energy-usage event to Kafka. The Usage Service consumes this event, writes timestamped usage data to InfluxDB, and evaluates threshold-based conditions. The User and Device services, along with persisted alert records, use MySQL. This separation permits the ingestion request to be acknowledged without waiting for downstream analytics or local mail capture.

B. Deterministic Energy Insights

The Insight Service is intentionally deterministic and is not represented as machine learning. For a selected D-day window with aggregate energy E, the implementation derives a daily rate E/D and a 30-day projection 30E/D. It applies a demonstration tariff (a fixed monthly charge and three energy slabs) to calculate indicative cost. Devices are ranked by their share of total energy. The standby routine estimates baseline demand using a configured 20% average-load heuristic and categorizes the estimated level. These outputs are traceable application transformations, not a validated forecast against a utility bill or an appliance-level standby measurement.

C. Supervised ML Pipeline

Offline anomaly-model development used smart_meter_data.csv. The retained CSV has 5,000 rows sampled at 30-minute intervals from 1 January 2024 to 14 April 2024, with five numeric consumption/context fields, a timestamp, and Anomaly_Label. The preprocessing implementation removes invalid timestamps, invalid numeric values, negative energy values, and complete duplicates; no such rows were removed in the retained data. It then chronologically sorts the records, encodes Normal as 0 and Abnormal as 1, and derives Hour, DayOfWeek, and ConsumptionDeviation = Electricity_Consumed − Avg_Past_Consumption. The model uses eight features: the five raw numeric fields plus the three engineered features.

A 70%/15%/15% chronological train/validation/test split was used. The decision threshold was selected by validation F1 across candidate thresholds. Logistic Regression with standardization served as a baseline. The selected Random Forest uses 300 trees, min_samples_leaf = 2, class_weight = balanced_subsample, and random_state = 42. The saved artifact is loaded by the inference service; runtime payloads that omit environmental/context fields receive stored median imputations. Since the source of Avg_Past_Consumption and label construction is not documented, the workflow should not be described as completely leakage-free or externally representative.

At runtime, the ML service consumes energy-usage events, generates an anomaly probability with the persisted model, and publishes anomaly-results for the Alert Service. A bounded retry/failure-topic path is implemented, but no dedicated retry or failure-topic experiment was retained; it is therefore described here as an implementation feature rather than an evaluated resilience result.

D. Alerting, Deduplication, and Monitoring

The Alert Service consumes both threshold energy-alerts and ML anomaly-results. Before persistence and notification, it forms a deduplication key userId:deviceId-or-HOUSEHOLD:alertType and checks whether a matching alert exists within the configured 15-minute window. This mechanism was exercised in a controlled duplicate-event case; it is not evidence of a general duplicate-reduction rate. Mailpit acts as a local SMTP capture environment, so an observed message demonstrates local sink capture rather than external email delivery.

Observability is provided by Spring Actuator and ML metrics exposed to Prometheus, with Grafana available for local dashboards. The final recorded snapshot contained eight active Prometheus targets, all reported as up. The implementation exposes counters for alert creation and suppression; their recorded values are cumulative local-process counters and must not be interpreted as per-experiment rates. Distributed tracing, Kafka-lag analysis, and measured ML-inference latency were not retained as evidence.

E. Operational Workflow

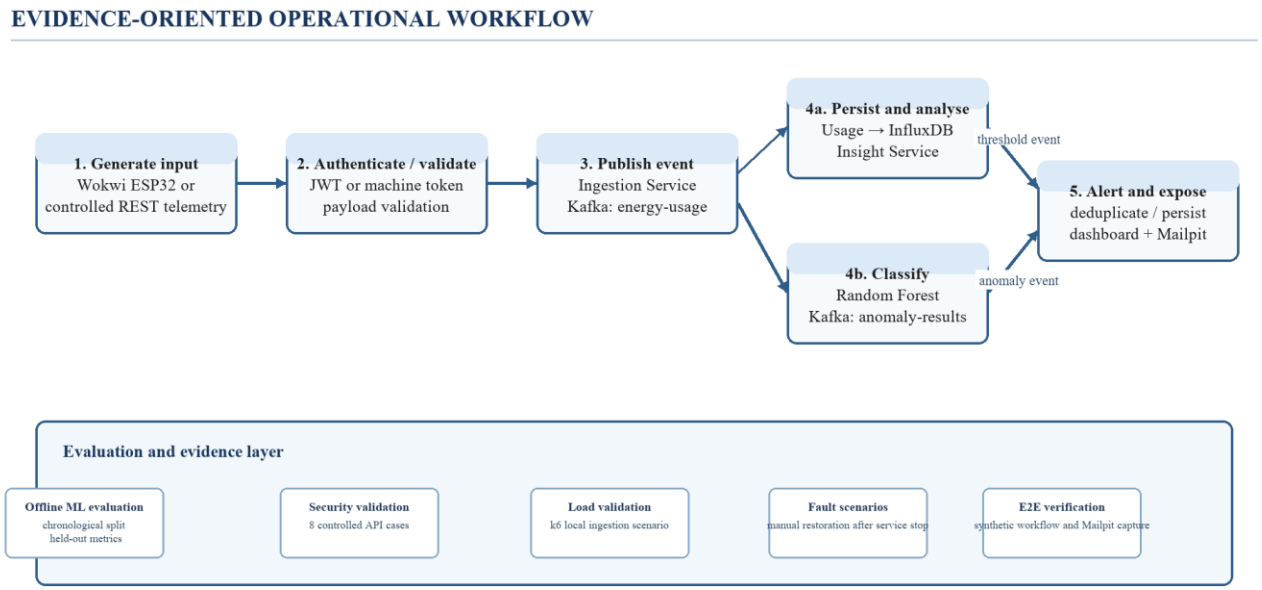


Fig. 2. Evidence-oriented workflow. Offline model evaluation, local functional tests, and runtime demonstrations are reported as distinct forms of evidence.

The workflow in Fig. 2 distinguishes processing from evaluation. The left-to-right path represents the implemented system; the lower band records how different parts were assessed. This distinction prevents a functional Kafka or Wokwi demonstration from being misreported as an ML accuracy study, a performance benchmark, or physical-meter validation.

Algorithm 1. Secure telemetry-to-alert event processing

```
Input: telemetry t = {deviceId, energyConsumed, timestamp}; caller credential c.
Output: accepted event status; downstream usage, insight, and alert artifacts when their asynchronous consumers complete.
1: If t is invalid or energyConsumed < 0, reject the request.
2: If c is neither a valid JWT nor the configured device token, return unauthorized.
3: Route the accepted request through the API Gateway to the Ingestion Service.
4: Publish t to Kafka topic energy-usage and return the ingestion acceptance status.
5: Usage consumer writes a time-series record and may publish a threshold event.
6: ML consumer builds the available feature vector, imputes missing optional context with stored medians, and publishes an anomaly-result event.
7: Alert consumer forms key userId:deviceId-or-HOUSEHOLD:alertType. If a matching alert lies inside the 15-minute window, suppress it; otherwise persist it and attempt local notification delivery.
```

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. Local Deployment and Software Components

TABLE I. Final implemented components and evidence boundaries

Layer	Implemented component(s)	Role in this study
Input	Wokwi ESP32 simulator; controlled REST client	Generate Wokwi-compatible synthetic telemetry; not a calibrated meter.
Identity and gateway	Keycloak; API Gateway	JWT role mapping, ownership checks, device-token telemetry route.
Event processing	Ingestion, Usage, ML Inference, Alert services; Kafka	Event distribution, time-series persistence, inference, threshold and anomaly handling.
Data	InfluxDB; MySQL	Usage series; user, device, and alert metadata.

Layer	Implemented component(s)	Role in this study
Presentation and notification	React frontend; Mailpit	Authenticated dashboard use and local mail capture.
Monitoring	Prometheus; Grafana; Actuator / ML metrics	Dated local health and metric observations.

TABLE II. Retained local experimental environment

Item	Recorded configuration	Scope
Host operating system	Windows 11 Home Single Language, build 26300	Host used for recorded local validation.
Compute record	8 logical processors; 8,311,836,672 bytes physical memory (about 7.74 GiB)	Hardware record captured with the k6 evidence.
Deployment	Docker Compose, 17 configured services	Local single-host deployment; not a distributed cluster.
Load tool	Grafana k6 Docker image	One specified ingestion scenario; no repeated-trial confidence interval.
Mail test endpoint	Mailpit	Local capture only; not an external mail-provider evaluation.

B. Dataset and Model Configuration

TABLE III. Dataset characteristics and split configuration

Characteristic	Verified value
Dataset identity	smart_meter_data.csv; source, licence, and label-generation provenance are absent from the repository.
Raw shape	5,000 rows; 7 CSV columns: Timestamp, Electricity_Consumed, Temperature, Humidity, Wind_Speed, Avg_Past_Consumption, Anomaly_Label.
Time coverage	2024-01-01 00:00:00 to 2024-04-14 03:30:00 at uniform 30-minute intervals.
Data-quality checks	0 missing raw values; 0 complete duplicate rows; 0 invalid timestamp/numeric/negative-energy rows in the retained data.
Class distribution	4,750 Normal (95.0%); 250 Abnormal (5.0%).
Chronological split	Train 3,500 (3,335/165); validation 750 (705/45); held-out test 750 (710/40), reported as Normal/Abnormal.
Feature set	Five raw numeric fields plus Hour, DayOfWeek, and ConsumptionDeviation.

TABLE IV. Candidate model configuration

Model	Configuration	Threshold selection
Logistic Regression	StandardScaler → LogisticRegression(max_iter=1000, class_weight=balanced, random_state=42)	Validation F1, 0.05–0.95 search in 0.01 increments.
Random Forest (selected)	300 trees; min_samples_leaf=2; class_weight=balanced_subsample; random_state=42; n_jobs=-1	Validation F1; selected threshold 0.31.

C. Test Procedures

1. Offline ML evaluation: audit the CSV, apply the chronological split and feature transformation, select the threshold by validation F1, and report untouched held-out test metrics.
2. Security validation: obtain local Keycloak tokens and execute eight expected-status scenarios for owned access, unauthorized access, ownership rejection, administration, machine token, and token expiry.
3. Functional/E2E verification: create a controlled profile and device, submit synthetic telemetry through the gateway, poll the downstream APIs, and compare deterministic results and Mailpit capture with predeclared expected values.

4. Alert deduplication: submit identical records, inspect alert persistence and the suppression log inside the configured 15-minute window.
5. Performance validation: run the recorded k6 ingestion profile (1 request/s start; 2, 5, and 10 requests/s for 30 s stages; 15 s ramp-down) using authenticated local ingestion requests.
6. Fault scenarios: stop Mailpit or the ML service, test whether gateway ingestion is accepted, manually restart the stopped service, and record container-state recovery. This procedure does not assert end-to-end replay or delivery.

V. RESULTS AND DISCUSSION

A. Offline ML Evaluation

Table V reports model results on the held-out chronological test interval. Random Forest was selected by validation F1 and substantially outperformed the Logistic Regression comparator on this specific retained interval. The Random Forest confusion matrix contains 705 true negatives, 5 false positives, 3 false negatives, and 37 true positives. Although an accuracy of 742/750 (98.93%) follows from this matrix, F1, PR-AUC, and recall are more informative in the 5% abnormal-class setting and are therefore emphasized.

TABLE V. Offline anomaly-detection results on the held-out CSV interval

Model	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.28	0.05333	1.00000	0.10127	0.42285	0.05971
Random Forest	0.31	0.88095	0.92500	0.90244	0.99743	0.94853

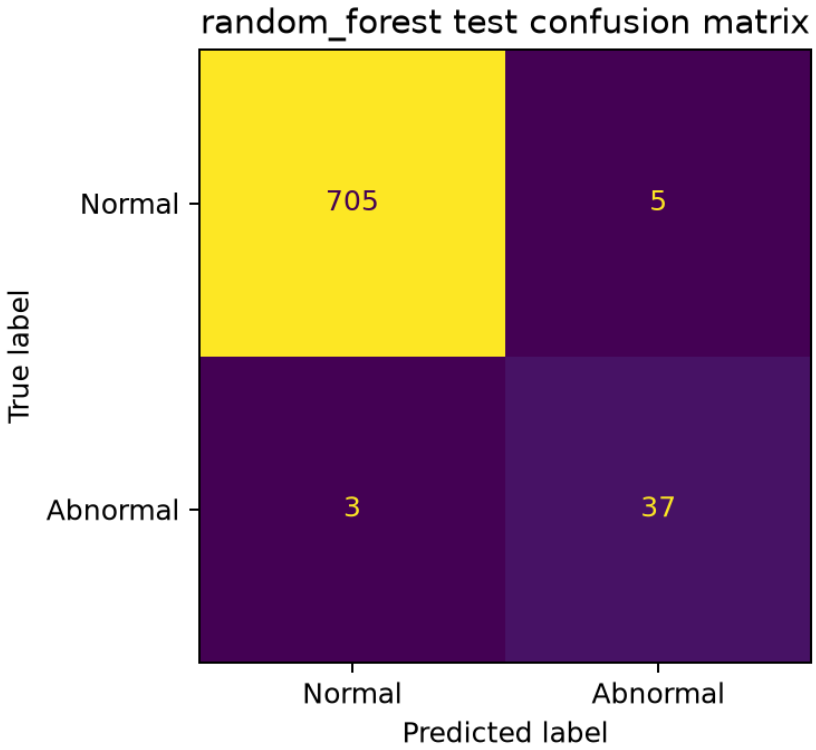


Fig. 3. Random Forest confusion matrix on the 750-record held-out CSV interval.

The metric results characterize one anonymous, labelled CSV stream. They do not establish physical deployment performance. In particular, the dataset does not document its source, licence, entity identity, or label-generation method. Moreover, the Wokwi modes emit 1–40 kWh values while the recorded training Electricity_Consumed range is 0–1 with undocumented units; optional contextual features are median-imputed at runtime. Consequently, the offline metrics must not be extrapolated to simulated heavy-load modes, household meters, or utility data.

B. Controlled Functional and End-to-End Verification

The latest successful E2E record used synthetic REST telemetry submitted through the Wokwi-compatible endpoint. Three analytics inputs (0.25, 0.50, and 0.75 kWh) resulted in a 1.50 kWh total. The documented forecast, ranking, and standby assertions passed. A separate 1.00 kWh input against a 0.10 kWh user threshold produced a persisted THRESHOLD alert with sent = true and a matching Mailpit message. The record also contained an ML_ANOMALY alert with probability 0.792945, but the E2E script's formal pass condition was the threshold alert and email, not prediction quality.

TABLE VI. Latest controlled E2E functional observations

Case	Predeclared / observed result	Interpretation and boundary
Analytics path	1.50 kWh; current estimate $0.18; daily rate 0.50 kWh; monthly projection 15.00 kWh and $11.80; 100% one-device ranking; 4.17 W standby estimate.	Configured deterministic transformations passed for the synthetic case; not bill or meter ground truth.
Analytics availability	3.79 s observation after submissions.	Polling-bounded local functional observation; not an end-to-end latency benchmark.
Threshold alert	1.00 kWh > 0.10 kWh; persisted THRESHOLD alert sent = true; matching Mailpit capture.	Local threshold-to-mail-sink path was observed.
Alert observation	13.64 s observation.	Includes polling; not SMTP or service latency.
Runtime ML observation	One ML_ANOMALY record with probability 0.792945 and threshold 0.31.	Integration observation only; not an E2E ML-quality assertion.

C. Gateway Security and Dashboard Regression Validation

The local security suite passed all eight configured cases on 5 October 2026. These results show that the expected HTTP behavior was observed for the tested gateway boundary paths. They are not a penetration test, cryptographic assessment, privacy assessment, or evidence of production security.

TABLE VII. Local gateway security validation

Control exercised	Expected / observed status	Result
JWT-owned profile access	200	Pass
Missing JWT	401	Pass
Invalid JWT	401	Pass
Cross-user profile access	403	Pass
Cross-user device-list access	403	Pass
Administrator user listing	200	Pass
Machine-token telemetry ingestion	201	Pass
Expired JWT	401	Pass

After a reactive gateway response-body correction, four focused regression tests passed and eight protected dashboard routes returned non-empty HTTP 200 JSON responses in the dated local runtime validation. Historical broader service tests (28 Java tests with 3 skipped) and 3 ML API tests are reported as earlier, separate runs; they are not combined into a falsely contemporaneous total.

D. Alert Deduplication and Observability

In the duplicate-event validation, two identical submissions were accepted by ingestion, one ML_ANOMALY alert was persisted, and an explicit suppression log was observed inside the 15-minute window. Unit tests also covered first-alert creation, inside-window suppression, post-window creation, and independent deduplication keys. This supports correct behavior for the tested scenario, but it does not quantify duplicate reduction under arbitrary retries, partitions, or concurrent consumers.

The final Prometheus snapshot reported eight active targets, each with health up, and local health checks returned HTTP 200 for the frontend, gateway health endpoint, Prometheus readiness endpoint, Grafana health endpoint, and Mailpit API. Cumulative suppression counters existed for ML_ANOMALY and THRESHOLD alert types. No derived alert rate, Kafka lag, distributed trace, or measured ML latency is reported because such evidence was not retained.

E. Local Load and Controlled Fault Results

TABLE VIII. Local load and controlled fault observations

Experiment	Observed result	Evidence-bound interpretation
k6 ingestion scenario	450/450 acceptance checks; 0.00% HTTP failure; mean 39.44 ms; median 24.85 ms; p90 74.78 ms; p95 109.77 ms; maximum 503.36 ms; mean 4.281 requests/s.	Single-host local validation under the recorded ramp to 10 requests/s; not a production capacity or distributed scalability result.
Mailpit service-stop scenario	Gateway ingestion returned HTTP 201 while Mailpit was stopped; the harness later restored the container and recorded recovered = true.	Shows recorded ingress availability and manual restoration only; it does not establish notification queuing, automatic recovery, or eventual mail delivery.
ML-service stop scenario	Gateway ingestion returned HTTP 201 while the ML service was stopped; the harness later restored the container and recorded recovered = true.	Shows recorded ingress availability and manual restoration only; it does not establish replay, post-restart inference, or alert delivery.

The k6 result demonstrates that the gateway accepted the recorded workload in the specified local configuration. It cannot establish multi-node scalability, cloud performance, resource efficiency, or behavior under a broker/database failure. Similarly, the two controlled service-stop scenarios show that synchronous gateway ingestion did not fail in the recorded cases; they do not demonstrate comprehensive fault tolerance.

F. Discussion of the Revised Contribution

The principal contribution is an implementation-level, evidence-bounded integration: event-driven telemetry processing with time-series persistence; deterministic user-facing analysis; a separately evaluated supervised anomaly classifier; gateway authentication and ownership controls; deduplicated alert handling; and local operational checks. The model comparison establishes that the selected Random Forest performed better than the baseline on the retained test interval, while the E2E run establishes a synthetic integration path rather than generalizable ML efficacy. The security, k6, and fault results support localized claims about the executed cases only.

VI. LIMITATIONS AND FUTURE WORK

Several limitations remain. First, Wokwi values and controlled REST inputs are synthetic and have not been calibrated against a physical energy meter. The demonstration tariff, ranking logic, and standby heuristic have not been validated against utility bills, device-level instrumentation, or user behavior. Second, smart_meter_data.csv has no documented source, licence, household identity, or label-generation procedure. Its consumption scale and units do not align demonstrably with all Wokwi modes, and runtime imputation of absent context limits interpretability. Third, all runtime work was performed on a single local Docker host using one load scenario and a short evidence window. Mailpit represents local message capture rather than external delivery.

Future research should obtain consented, calibrated, multi-household energy data with documented labels and provenance; align live measurement units and contextual features with model training; assess temporal drift and external generalization; validate forecasts and tariff calculations against bills; and investigate appliance-level disaggregation. Systems work should evaluate multi-node deployment, durable retry/DLQ behavior, broker/database failures, longer-duration monitoring, stronger device provisioning, TLS/cloud configurations, distributed tracing, and repeated performance experiments with confidence intervals.

VII. CONCLUSION

This paper presents the final, evidence-based form of a Home Energy Tracker built as an event-driven local microservices system. The implementation integrates Wokwi-compatible telemetry, gateway JWT and device-token controls, Kafka, InfluxDB, MySQL, deterministic insights, a persisted Random Forest anomaly service, deduplicated alerts, and Prometheus/Grafana monitoring. The retained evidence includes a chronological offline ML evaluation, eight expected security outcomes, a controlled alert-deduplication case, an eight-target Prometheus snapshot, a 450-request local ingestion scenario, two controlled service-stop records, and a successful synthetic end-to-end workflow with local Mailpit capture. These results support claims of local integration and bounded evaluation. They do not establish physical-meter accuracy, utility-bill validity, external ML generalization, production security, production capacity, or comprehensive resilience. The most defensible next step is therefore field-quality data and evaluation rather than stronger claims from the present local prototype.

REFERENCES

[1] A. Shuaibu, S. U. Haq, B. Almadani, F. Aliyu, and E. Al-Nahari, "Smart Metering Meets AI: Real-Time Appliance Monitoring and Anomaly Detection Over DDS Middleware," IEEE Transactions on Industry Applications, 2026, doi: 10.1109/TIA.2025.3608687.

[2] I. Núñez, C. A. Rovetto, E. Cruz, A. Smolarz, D. H. Concepción, and E. Cano, "Design of a microservices-based architecture for residential energy efficiency monitoring," International Journal of Electronics and Telecommunications, 2024, doi: 10.24425/ijet.2024.152511.

[3] W. Agustiarmi et al., "Real-Time IoT-based Energy Power Consumption Monitoring System for Smart Homes," International Journal on Advanced Science, Engineering and Information Technology, 2025, doi: 10.18517/ijaseit.15.5.20918.

[4] M. Islam, N. Jobaida, M. S. Hossain, M. R. Uddin, and M. Hasan, "IoT-Based Smart Energy Monitoring and Energy Demand Response System," 2024 IEEE 3rd International Conference on Electrical Power and Energy Systems (ICEPES), 2024, doi: 10.1109/ICEPES60647.2024.10653605.

[5] S. S. Shashank, Yogesh, M. Krishnamoorthy, N. Karuppiah, P. Mounica, and R. Abinaya, "Smart Energy Monitoring for Real-Time Power Consumption Tracking and Optimization Using an IoT-Based Cost-Effective System," 2025 11th International Conference on Electrical Energy Systems (ICEES), 2025, doi: 10.1109/ICEES67011.2025.11212888.

[6] T. B. D. Sousa and R. Rabelo, "Smart Metering Enabled by the Internet of Things for Monitoring Residential Energy Consumption and Power Quality," IEEE Access, 2025, doi: 10.1109/ACCESS.2025.3627191.

[7] B. Jyothi et al., "Empowering energy management: anomaly detection in smart meter data for proactive consumption control," Bulletin of Electrical Engineering and Informatics, 2026, doi: 10.11591/eei.v15i3.10957.

[8] K. Kulkarni, "Anomaly Detection in Smart Electric Meters for Detecting Faults and Misuse of Electric Energy Consumption," International Journal of Scientific Research in Engineering and Management, 2024, doi: 10.55041/ijsrem34689.

[9] M. AbuArqoub, A. A. Titinchi, and R. Marie, "A Scalable IoT-AI Framework for Real-Time Energy Anomaly Detection in Smart Grid Networks," 2026 2nd International Conference on Computational Intelligence Approaches and Applications (ICCIAA), 2026, doi: 10.1109/ICCIAA68481.2026.11543846.

[10] Ó. Hernández et al., "Detection of Anomalies in Daily Activities Using Data from Smart Meters," Sensors, vol. 24, no. 2, 2024, doi: 10.3390/s24020515.

[11] S. Maitra, S. Kundu, and A. Shankar, "A Real-time Anomaly Detection Using Convolutional Autoencoder with Dynamic Threshold," arXiv, 2024, doi: 10.48550/arXiv.2404.04311.

[12] A. F. S. Veloso, J. Reis, and R. Rabelo, "A Microservices-Based Solution with Hybrid Communication for Energy Management in Smart Grid Environments," Sensors, 2026, doi: 10.3390/s26051714.

[13] D. K, "Smart Energy Monitoring System," International Journal of Innovative Research in Information Security, 2025, doi: 10.26562/ijiris.2025.v1109.15.

[14] L. L. Motta, W. S. Alvarez, H. R. Carvajal, F. B. Neto, E. D. Lima, and L. F. P. Meloni, "A cloud architecture for home energy management systems: a conceptual model," Energy Informatics, 2025, doi: 10.1186/s42162-025-00599-1.

[15] J. Liu and Y. Hsiao, "Smart plug-based home energy management with IoT and anomaly detection," Journal of the Chinese Institute of Engineers, 2025, doi: 10.1080/02533839.2025.2557233.

[16] K. M. W. Hidayat and M. G. Al-Faris, "IoT-Based Electrical Power Consumption Monitoring System in Households Using ESP32 and PZEM-004T," Brilliance: Research of Artificial Intelligence, 2025, doi: 10.47709/brilliance.v5i2.6368.

[17] S. Sirisumrannukul, T. Intaraumnauy, and N. Piamvilai, "Optimal control of cooling management system for energy conservation in smart home with ANNs-PSO data analytics microservice platform," Heliyon, 2024, doi: 10.1016/j.heliyon.2024.e26937.

[18] M. S. Hossain and M. Hassan, "Smart Energy Meter with Real Time Monitoring System," 2024 13th International Conference on Electrical and Computer Engineering (ICECE), 2024, doi: 10.1109/ICECE64886.2024.11024970.

[19] D. G. S, T. Gangakhedkar, A. P, A. Ashwija, and C. B. T. Rambapuri, "IOT-Enabled Electrical Consumption Tracking for Intelligent Energy Saving," International Journal of Scientific Research in Engineering and Management, 2025, doi: 10.55041/ijsrem54847.

[20] Y. Zhu and M. Yue, "Design of an IoT-Based Smart City Energy Management System," 2025 4th International Conference on Green Energy and Power Systems (ICGEPS), 2025, doi: 10.1109/ICGEPS65133.2025.11034388.

[21] S. Susikala, R. K, J. K, S. K, and Y. M, "IoT-Based Smart Electricity Monitoring System with Real-Time Alerts and Remote Control," International Journal of Engineering & Extended Technologies Research, 2026, doi: 10.15662/ijeetr.2026.0802420.

[22] C. S. Sinha, "Smart Meter-Based Power Consumption Detection System: An Integrated Approach Using LSTM Networks and Real-Time Web Interface," International Journal For Multidisciplinary Research, 2025, doi: 10.36948/ijfmr.2025.v07i01.37948.

[23] F. Dinmohammadi, A. M. Farook, and M. Shafiee, "Improving Energy Efficiency in Buildings with an IoT-Based Smart Monitoring System," Energies, 2025, doi: 10.3390/en18051269.

[24] S. Son, S. Ko, H. Lee, S. Lee, and B. Lee, "SIMA: Scalable IoT Microservice Architecture With Users and Things Joint Load Balancing," IEEE Internet of Things Journal, 2024, doi: 10.1109/JIOT.2023.3317430.